Universidade do Minho
Escola de Engenharia

Relatório 1ª Fase | Laboratórios de Informática III

Grupo 52 | 2025/26

Cristiano Gomes (A111836) Duarte Rodrigues (A111517) Mariana Leal (A111248)

Índice

1. **Introdução** 3

2. **Sistema** 4

2.1. Managers 5

2.2. Parsing 5

2.3. Queries 5

3. **Discussão** 6

3.1. Modularização e Encapsulamento 6

3.2. Estrutura de dados 6

3.3. Execução das Queries 7

3.3.1. Querie 1 7

3.3.2. Querie 2 7

3.3.3. Querie 3 7

4. **Conclusão** 8

1. Introdução

O presente relatório apresentará informações relativas à 1ª Fase do Projeto Final da Unidade Curricular Laboratórios de Informática III, pertencente ao 2º Ano da Licenciatura em Engenharia Informática, realizada no ano letivo 2025/2026, na Universidade do Minho.

O projeto desenvolvido nesta fase consiste na criação de uma base de dados em memória, construída a partir dos ficheiros .csv fornecidos, que incluem informação sobre aeroportos, voos, aeronaves, passageiros e reservas. O objetivo é carregar, validar e armazenar estes dados e permitir a execução de operações de pesquisa e associação entre as diferentes entidades, de forma a responder às queries definidas no enunciado.

Neste trabalho são aplicados conceitos fundamentais como modularidade, encapsulamento, reutilização e abstração de dados.

Na 1ª fase, foi necessário implementar o parsing e a validação dos ficheiros de entrada, o modo batch através do *programa-principal*, e a execução das três queries obrigatórias definidas no enunciado.

2. Sistema

A arquitetura do sistema desenvolvido nesta fase foi organizada de forma modular, separando claramente os componentes responsáveis pela leitura, validação, armazenamento e consulta dos dados. O programa foi estruturado em diversos módulos independentes, permitindo uma gestão eficiente das entidades (aeroportos, voos, aeronaves, passageiros e reservas) e garantindo a flexibilidade necessária para evoluções futuras do projeto.

No projeto deste ano foram implementados o parsing e a validação dos dados de entrada, bem como o modo batch através do **programa-principal**, responsável por interpretar o ficheiro de comandos e executar sequencialmente as queries definidas. Este processo envolve não só a leitura e interpretação dos diferentes ficheiros CSV, mas também o armazenamento organizado dos dados em memória, recorrendo a estruturas adequadas para garantir eficiência no acesso a cada entidade.

Para suportar esta organização, foi utilizada a biblioteca **glib**, que disponibiliza estruturas de dados robustas — como tabelas de hash e listas ligadas — facilitando a gestão, consulta e indexação da informação carregada.

No desenvolvimento do trabalho, utilizou-se a seguinte estrutura:

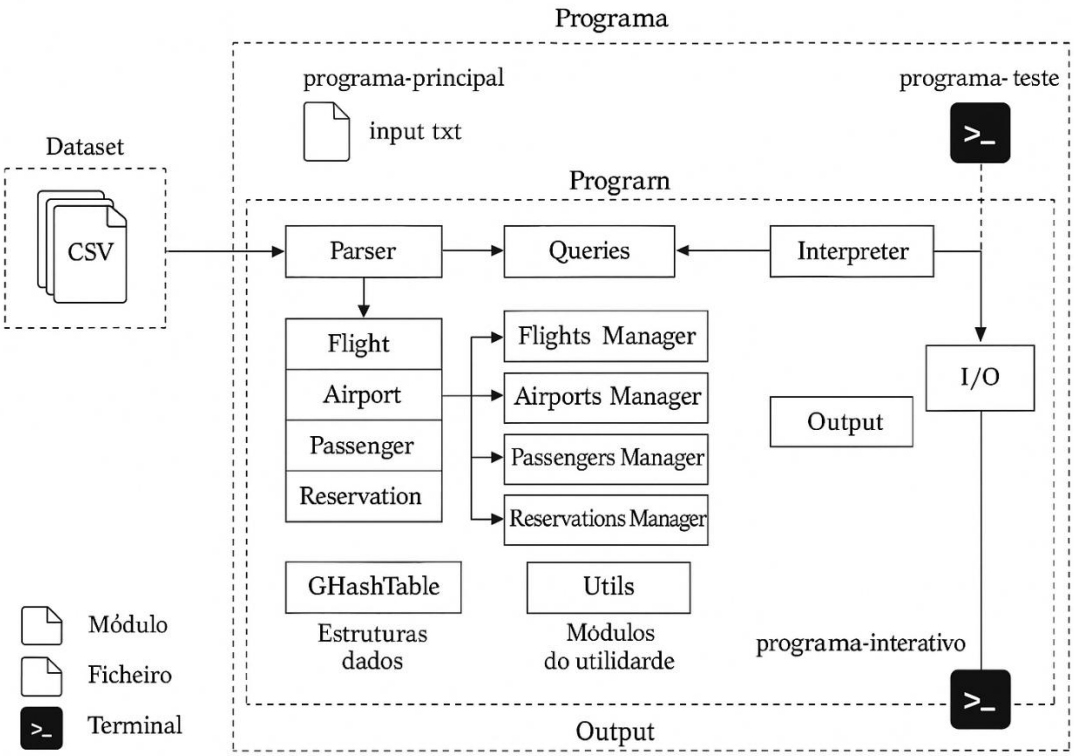

Figure 1: Arquitetura da Aplicação.

2.1. Managers

Os *managers* são os módulos responsáveis por armazenar e gerir os dados carregados dos ficheiros CSV. Cada entidade do sistema — voos, aeroportos, aeronaves, passageiros e reservas — possui o seu próprio manager, que utiliza principalmente tabelas de hash da biblioteca **glib** para organizar a informação em memória e permitir operações eficientes de inserção e consulta.

2.2. Parsing

Os parsers do projeto estão organizados por entidade, existindo um módulo específico para cada ficheiro CSV do dataset: *flights_parser*, *airports_parser*, *aircrafts_parser*, *passengers_parser* e *reservations_parser*. Cada um deles utiliza um parser genérico para ler o ficheiro linha a linha, dividir os campos e preencher uma estrutura temporária correspondente ao tipo de dados em processamento.

Durante esta fase são aplicadas todas as validações sintáticas exigidas no enunciado, assegurando que datas, códigos IATA, formatos numéricos, listas, URLs e restantes campos apresentam o formato correto. Sempre que necessário, os valores são normalizados removendo aspas, convertidos para os tipos adequados e preparados para a validação seguinte.

Após a validação sintática, cada parser executa a validação lógica específica da entidade — por exemplo, garantir que um voo existe antes de ser associado a uma reserva, que uma aeronave indicada num voo está registada, que horários reais são consistentes com horários estimados ou que listas de voos numa reserva contêm apenas voos válidos e corretamente encadeados. Em caso de erro sintático ou lógico, a linha original é escrita no respetivo ficheiro *_errors.csv*, garantindo um registo completo das entradas inválidas.

2.3. Queries

Os módulos query1, query2 e query3 são responsáveis pela execução das interrogações definidas no enunciado da fase 1. Cada query recebe os seus argumentos já processados pelo parser de comandos e obtém acesso às estruturas de dados necessárias, nomeadamente às hash tables e arrays mantidos pelos managers das diferentes entidades. Desta forma, cada módulo pode consultar eficientemente aeroportos, voos, aeronaves, passageiros ou reservas, consoante os requisitos da query.

3. Discussão

3.1. Modularização e Encapsulamento

Nesta 1.ª fase do trabalho, iniciámos por definir a arquitetura do sistema, o que tornou a implementação muito mais organizada. A separação entre *managers* e entidades contribuiu significativamente para a clareza e manutenção do código, evitando dependências diretas entre módulos. Cada manager passou a ser responsável pela lógica de armazenamento e acesso aos dados — como voos, aeroportos, aeronaves, passageiros e reservas — enquanto as entidades representam apenas as estruturas que armazenam a informação. Esta divisão permitiu um encapsulamento claro, mantendo as entidades simples e concentrando a complexidade das operações nos managers, o que facilita a evolução e expansão futura do sistema.

A modularidade trouxe também alguns desafios, sobretudo ao nível da consistência entre interfaces. À medida que o número de módulos cresceu, aumentou a necessidade de garantir que todas as funções expostas pelos headers permaneciam coerentes. Pequenas alterações numa interface exigiam ajustes imediatos nos módulos dependentes, obrigando a um planeamento cuidadoso e a uma comunicação clara entre componentes para evitar erros e garantir a correta integração de todas as partes do sistema.

3.2. Estrutura de dados

As estruturas de dados utilizadas nesta fase foram principalmente a **GHashTable** e o **GArray**, disponibilizadas pela biblioteca *GLib*. Estas estruturas mostraram-se particularmente vantajosas para um projeto desta dimensão, onde é necessário armazenar e consultar grandes volumes de informação provenientes dos ficheiros CSV. A *GLib* facilita o processo ao fazer a gestão dinâmica da memória, permitindo redimensionamento automático e fornecendo funções de libertação adequadas, reduzindo significativamente o risco de *memory leaks*. Para além disso, disponibiliza funções utilitárias de iteração e procura que simplificam o desenvolvimento e tornam o código mais robusto.

Apesar de existirem outras abordagens possíveis, optámos por centrar a implementação nestas duas estruturas. O **GArray** mostrou-se útil para armazenar conjuntos de elementos de forma contígua, permitindo acesso direto eficiente. Já a **GHashTable** foi essencial para representar coleções grandes indexadas por identificadores únicos, como aeroportos, voos, aeronaves ou passageiros, permitindo pesquisas rápidas em tempo próximo de O(1). Embora haja espaço para explorar estruturas mais avançadas no futuro, estas escolhas revelaram-se adequadas para as necessidades da fase 1.

3.3. Execução das *Queries*

3.3.1. Querie 1

A ***Query* 1** recebe o código IATA de um aeroporto e verifica a sua existência no *AirportsManager*. A pesquisa é feita diretamente na tabela de *hash* onde os aeroportos válidos foram previamente inseridos durante o *parsing* dos *CSV*. Se o aeroporto existir, o sistema recolhe os seus campos principais — código, nome, cidade, país e tipo — e constrói a linha final no formato exigido pelo enunciado. Caso contrário, devolve simplesmente uma linha vazia, uma vez que o código não corresponde a nenhum aeroporto válido carregado.

3.3.2. Querie 2

A ***Query* 2** percorre todas as aeronaves presentes no *AircraftsManager* e calcula, para cada uma delas, o número total de voos não cancelados registados no *FlightsManager*. Para isso, o sistema itera sobre todos os voos armazenados e compara o identificador da aeronave de cada voo com o identificador atual. Caso seja fornecido um fabricante como filtro opcional, o sistema ignora todas as aeronaves cujo fabricante não corresponda ao argumento recebido. Depois de obtidos os contadores, é efetuada uma ordenação: primeiro por número de voos (de forma decrescente) e, em caso de empate, por ordem lexicográfica crescente do identificador. Os primeiros N elementos da lista ordenada são então convertidos no formato final contendo identificador, fabricante, modelo e número de voos realizados.

3.3.3. Querie 3

A ***Query* 3** determina qual o aeroporto com mais partidas dentro de um intervalo de datas, analisando exclusivamente os voos cuja data real de partida (*actual departure*) esteja compreendida no período definido. Para comparar datas, é utilizada uma comparação direta com strncmp, uma vez que o formato YYYY-MM-DD permite ordenação lexicográfica. Apenas voos com estado diferente de *Cancelled* são considerados. Durante o processamento, é mantido um contador para cada aeroporto de origem, acumulando o número de partidas válidas. Depois de percorrer todos os voos, o sistema escolhe o aeroporto com maior número de partidas e, em caso de empate, seleciona o código IATA menor por ordem lexicográfica. A linha final inclui o código, nome, cidade, país e número total de partidas. Se não existir nenhum voo válido dentro do intervalo, é devolvida uma linha vazia.

4. Conclusão

Com a realização deste projeto aprofundámos, na prática, os conceitos de **modularidade**, **organização do código** e **validação de dados**, que foram aplicados de forma consistente ao longo do desenvolvimento da primeira fase. Estes princípios revelaram-se essenciais para estruturar um sistema robusto, permitindo separar responsabilidades entre módulos, facilitar a leitura do código e garantir reutilização futura. Também reforçámos a importância de uma boa gestão de memória, sobretudo num projeto em C, onde a ausência de *memory leaks* e a utilização eficiente dos recursos têm impacto direto na estabilidade e desempenho do programa.

Durante o desenvolvimento apercebemo-nos de que existem aspetos que poderão ser melhorados, especialmente na parte do **parsing e das validações**, que apesar de funcionarem corretamente, poderão ser otimizados tanto em termos de tempo de execução como de utilização de memória. Certas operações, como a contagem de voos ou a filtragem por datas, poderão também beneficiar de estruturas de dados mais eficientes ou de estratégias de pré-processamento.

Estamos conscientes de que a **2.ª fase** irá representar um desafio adicional: o dataset será maior, o modo interativo exigirá maior cuidado com desempenho, e as queries restantes serão mais complexas e intensivas na forma como acedem e relacionam informação. Será, por isso, necessário garantir uma abordagem eficiente, minimizar acessos redundantes e assegurar que o programa continua rápido e seguro, mesmo com um volume de dados significativamente superior.

No geral, esta fase permitiu-nos consolidar competências fundamentais e preparar uma base sólida para a continuação do projeto.